Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский ядерный университет «МИФИ»»

Институт интеллектуальных кибернетических систем
Кафедра № 42

ОТЧЕТ

по сессионному заданию
по дисциплине «Основы криптографии»

Выполнил: студент группы М26-545 Голубев А.Т.
Проверил: _____

Москва — 2026

СОДЕРЖАНИЕ

1. Трёхсторонний протокол распределения ключей между парами участников с использованием третьей стороны

2. Формальные модели шифров

3. Принципы построения поточных систем шифрования

4.Пример режима шифрования, в котором биективность шифрующих отображений не является необходимой для однозначного расшифрования

5.Распространение ошибки в режиме гаммирования с обратной связью по шифртексту (CFB)

6. Коллизия случайной пары сообщений при заданной вероятности

1. Трёхсторонний протокол распределения ключей между парами участников с использованием третьей стороны

Если каждой паре абонентов сети из N участников заранее выдавать отдельный секретный ключ, потребуется $N(N-1)/2$ ключей; уже при $N = 1000$ это 499 500 пар, хранить и заменять которые практически невозможно. Выход — привлекать доверенную третью сторону, центр распределения ключей: тогда каждый абонент заранее получает лишь один долговременный ключ, общий с центром, а сеансовый ключ для конкретной пары вырабатывается по запросу в протоколе, где участвуют все три стороны. Классической реализацией этой идеи является протокол Нидхэма–Шрёдера на симметричных ключах (1978 г.).

Роли сторон. A — инициатор сеанса: желает защитить обмен данными с абонентом B и владеет долговременным ключом K_AT, общим с центром. B — отвечающая сторона: владеет долговременным ключом K_BT, общим с центром. T — доверенный центр: хранит долговременные ключи всех абонентов, по запросу создаёт случайный сеансовый ключ K_AB и доставляет его сторонам только внутри шифртекстов; центр считается честным и не разглашает ключи.

Обозначения: E(K; M) — зашифрование сообщения M на ключе K; N_A и N_B — одноразовые случайные числа (нонсы), которые генерируют сами участники. Протокол состоит из пяти шагов.

Шаг 1. A передаёт центру T открыто тройку (A, B, N_A) — свой идентификатор, идентификатор абонента, с которым хочет связаться, и свежий нонс. По нонсу A позже отличит ответ именно на свой текущий запрос от повтора старого сообщения, которое мог перехватить противник.

Шаг 2. Центр T создаёт сеансовый ключ K_AB и посылает A сообщение E(K_AT; N_A, B, K_AB, E(K_BT; K_AB, A)). Внутренняя часть E(K_BT; K_AB, A) называется **билетом**: она зашифрована ключом K_BT, поэтому доступна

только B, а всё сообщение целиком закрыто ключом K_AT и читается только A. Расшифровав ответ, A убеждается, что её нонс N_A присутствует (ответ свежий), а идентификатор B указывает, что ключ предназначен именно для связи с B.

Шаг 3. A передаёт B билет E(K_BT; K_AB, A), который сама прочитать или незаметно изменить не может. B расшифровывает билет своим долговременным ключом, извлекает сеансовый ключ K_AB и имя партнёра; так как K_BT знают только B и центр, B заключает, что ключ выработан центром для связи с A.

Шаг 4. B вырабатывает нонс N_B и посылает A шифртекст E(K_AB; N_B) — запрос на подтверждение того, что A действительно владеет сеансовым ключом.

Шаг 5. A расшифровывает N_B, применяет к нему заранее оговорённое преобразование (например, вычитает единицу) и возвращает B шифртекст E(K_AB; N_B − 1). Преобразование необходимо, чтобы противник не мог скопировать шифртекст шага 4 в качестве ответа. Получив корректное значение, B убеждается, что A владеет ключом и участвует в текущем сеансе.

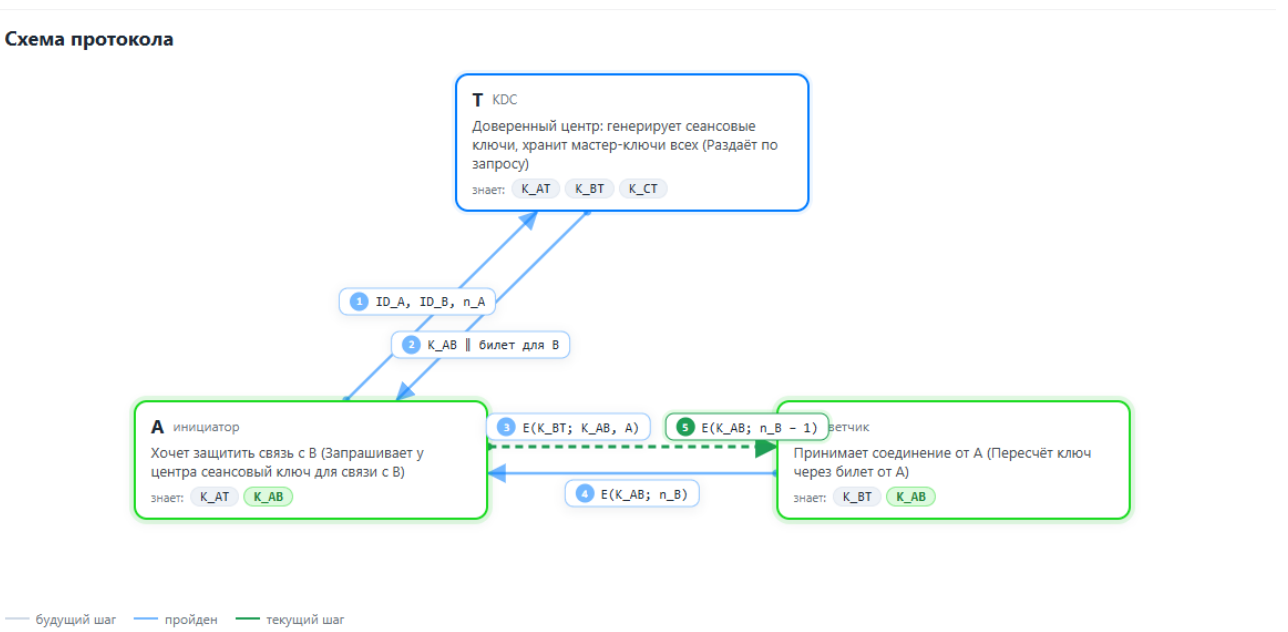
Итоговый результат. Между A и B установлен общий секретный сеансовый ключ K_AB, сгенерированный центром T. Ключ доступен только A, B и центру, а по открытому каналу передавался лишь внутри шифртекстов на долговременных ключах K_AT и K_BT; шаги 4–5 дают B подтверждение владения ключом абонентом A — но не подтверждение свежести самого ключа, и это различие играет ключевую роль в описанной ниже атаке. Далее стороны защищают с помощью K_AB данные сеанса, по окончании которого ключ уничтожается; для любой другой пары абонентов процедура повторяется.

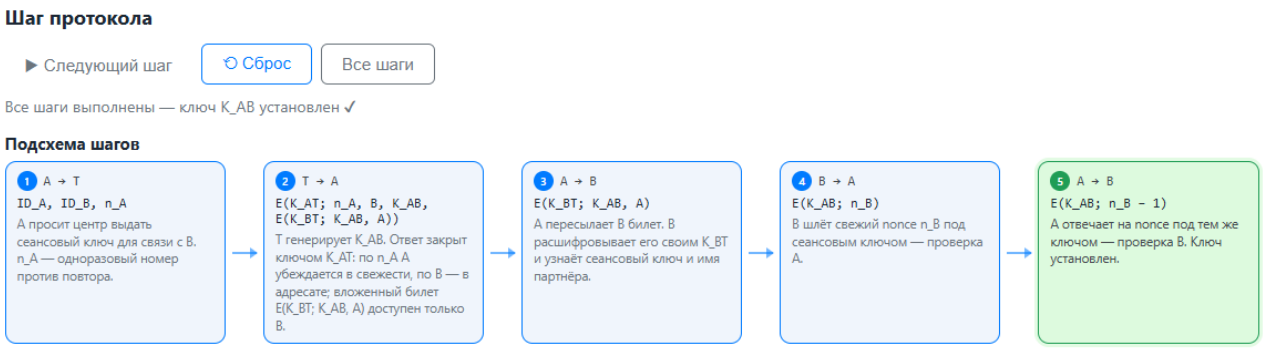
Недостатки схемы. Центр должен быть постоянно доступен и пользоваться полным доверием. Кроме того, B не может проверить свежесть билета. Билет передавался по открытому каналу, поэтому противник мог

записать его заранее; если когда-либо в прошлом он узнал старый сеансовый ключ K′ (в том числе вовсе не дожидаясь нового запуска протокола), он сам инициирует обмен с шага 3: повторно отправляет B сохранённый билет E(K_BT; K′, A) и правильно отвечает на шаге 5, выдавая себя за A (атака Деннинг–Сакко, 1981 г.). B при этом получает не свежий ключ центра, а старый, известный противнику.

Варианты защиты. Деннинг и Сакко предложили включать в сам билет метку времени и срок действия: при расшифровании билета B проверяет эти поля и отвергает просроченные — так устроен Kerberos. Сами Нидхэм и Шрёдер в исправленной версии 1987 г. пошли другим путём: добавлен предварительный обмен между A и B, в котором B вырабатывает собственный нонс N_B′ и передаёт его A; далее N_B′ попадает в запрос к центру, и центр включает его внутрь билета E(K_BT; K_AB, A, N_B′). Расшифровав билет, B сравнивает нонс со своим — по совпадению отличает свежий билет от повтора. Существует и самостоятельный протокол того же назначения — протокол Отвея–Рииса: там инициатор A сначала обращается к B, и именно B пересылает центру свою часть запроса (центр общается с обеими сторонами, а не только с A, как в Нидхэме–Шрёдере); свежесть ключа проверяют обе стороны по своим нонсам внутри ответа центра.

Дополнительно была разработана визуальная интерактивная схема работы протокола (рисунок 1) на ***js, html, css*** *для закрепления материала. Название файла:* **1q_key_distribution_protocol.html**





Роли участников

A (инициатор) — запрашивает у центра сеансовый ключ для связи с B

B (ответчик) — получает ключ через билет от A

T (KDC) — доверенный посредник: генерирует ключ и раздаёт его паре через мастер-ключи

Трассировка сообщений

Шаг	Отправитель	Получатель	Сообщение	Комментарий
1	A	T	`ID_A, ID_B, n_A`	запрос ключа, открытый канал
2	T	A	`E(K_AT; n_A, B, K_AB, E(K_BT; K_AB, A))`	ответ центра + билет для B
3	A	B	`E(K_BT; K_AB, A)`	билет для B
4	B	A	`E(K_AB; n_B)`	проверка A владельцем K_AB
5	**A**	**B**	**`E(K_AB; n_B − 1)`**	**проверка B, K_AB установлен**

Итог: между A и B установлен общий секретный сеансовый ключ K_AB, известный только им и центру T. Обе стороны подтвердили свежесть ключа и подлинность друг друга. Повторив протокол для пар (A, C) и (B, C), центр обеспечивает ключами все три пары участников. K_AB установлен ✓

Слабости схемы:

- T — единая точка доверия и отказа; компрометация мастер-ключа участника раскрывает все его сессии.
- B не может проверить свежесть билета — **атака Деннинг–Сакко**: если противник когда-либо узнал старый сеансовый ключ K′, он повторно отправляет B сохранённый билет `E(K_BT; K′, A)` и корректно отвечает на шаге 5, выдавая себя за A.

Защита: метка времени и срок действия в билете (Kerberos) либо предварительный обмен, где нонс B через A попадает в билет (Нидхэм–Шрёдер, 1987).

Рисунок 1 – Визуальная интерактивная схема работы протокола

2. Формальные модели шифров

Для строгого анализа шифр задают как математический объект, к которому применимы доказательства; основы такого описания заложил К. Шеннон. Используются несколько взаимодополняющих моделей: алгебраическая и вероятностная как основные, а также модели симметричного и асимметричного шифров, блочного шифра и автоматная модель. Каждая выделяет свой аспект — корректность, стойкость, устройство ключей или способ обработки данных.

1 Алгебраическая модель

Определение. Шифр — совокупность объектов вида

$$\Sigma = (X, K, Y, E, D) \tag{2.1}$$

где X — конечное множество открытых текстов; Y — конечное множество криптограмм; K — множество ключей, достаточно большое, чтобы полный перебор был неосуществим; E = $\{E_k\}$ и D = $\{D_k\}$ — семейства правил зашифрования (X → Y) и расшифрования (Y → X).

Центральное требование — однозначное расшифрование:

$$D_k(E_k(x)) = x \tag{2.2}$$

для любых k ∈ K, x ∈ X. Отсюда инъективность каждого E_k и |Y| ≥ |X|. При X = Y шифр эндоморфный: каждое E_k — подстановка X, и шифр задаётся таблицей зашифрования |K| × |X|, каждая строка которой — перестановка X.

Пример. Шифр Цезаря со сдвигами: X = Y = {буквы}, K = {0, 1, …, 25}; правила зашифрования и расшифрования задаются формулами (2.3) и (2.4). Модель корректна, но тривиально вскрывается перебором 26 ключей.

$$E_k(x) = (x + k) \bmod 26 \tag{2.3}$$

$$D_k(y) = (y - k) \bmod 26 \tag{2.4}$$

Назначение. Модель формализует корректность — расшифрование всегда восстановит текст. Стойкость она не описывает: знания противника в ней не учитываются.

2 Вероятностная модель

Алгебраическую модель дополняют распределениями: Σ_B = (X, K, Y, E, D, P(X), P(K)). Здесь P(X) отражает статистику текстов (её задают, например, цепью Маркова) и априорные сведения противника; P(K) — вероятностный механизм выбора ключа, обычно равновероятный. Совместное распределение текста и ключа порождает распределение на криптограммах:

$$P(y) = \Sigma_{(x,k):\, E_k(x) = y} P(x) \cdot P(k) \qquad (2.5)$$

По формуле Байеса определяются апостериорные вероятности P(x | y). В этой модели Шеннон определил совершенную секретность: для всех x ∈ X и y ∈ Y выполняется

$$P(x|y) = P(x) \qquad (2.6)$$

Криптограмма не сообщает противнику ничего нового. Необходимое условие совершенной секретности:

$$|K| \geq |X| \qquad (2.7)$$

Классический пример — шифр Вернама с однократной равновероятной гаммой. Здесь же вводятся энтропия ключа и расстояние единственности.

Пример. В шифре Цезаря с равновероятным ключом из 26 сдвигов P(x | y) ≠ P(x) — частотный анализ даёт противнику информацию. В Вернаме с ключом длины текста равенство (2.6) выполняется точно.

Назначение. Модель превращает «противник ничего не узнает» в проверяемое равенство и даёт теоретический предел стойкости.

3 Модели симметричного и асимметричного шифров

Различие — в устройстве ключей. В симметричном ключ расшифрования совпадает с ключом зашифрования или легко из него вычисляется, поэтому его нужно заранее доставить по защищённому каналу (AES, ГОСТ); к нему применима вероятностная модель. В асимметричном ключ — пара (k_e, k_d): k_e открыт, k_d секретен.

Требования: восстановление k_d по k_e и восстановление x по паре (y, k_e) вычислительно трудны; в основе — односторонние функции с секретом, стойкость оценивается теорией сложности вычислений.

Пример. RSA: зашифрование

$$c = m^e \bmod n \tag{2.8}$$

расшифрование

$$m = c^d \bmod n \tag{2.9}$$

где m — открытый текст, c — криптограмма; секрет — разложение модуля n на простые множители: оно даёт φ(n), по которому вычисляется d. Учебная иллюстрация: $n = 3 \cdot 11 = 33$, $e = 3$, $d = 7$.

Назначение. Модель описывает распределение ключевого материала: кто какими ключами владеет и что можно передавать по открытому каналу. Для симметричного шифра она фиксирует проблему предварительной доставки ключа (решается протоколом из раздела 1), для асимметричного — возможность открытого ключа зашифрования и подписи; стойкость здесь уже не «совершенность», а вычислительная трудность обращения односторонней функции с секретом.

4 Модель блочного шифра

Здесь X = Y = V_n — двоичные векторы длины n, K = V_m; каждый ключ задаёт подстановку на V_n, шифр — подмножество симметрической группы мощности не более 2^m. Сообщение режется на блоки, обрабатываемые в одном из режимов (замена, сцепление блоков).

Пример. AES: блоки по 128 бит, ключи 128/192/256 бит.

Назначение. Эталон качества — случайная подстановка на V_n: статистические отличия от неё дают противнику опору для анализа. Выбрать такую подстановку напрямую невозможно, поэтому практические шифры строят её композиционно — раундами перестановок и замен.

5 Автоматная модель

Шифр, обрабатывающий текст знак за знаком, описывается автоматом (I, S, O, h, f): входной алфавит, состояния, выходной алфавит, функции переходов и выходов. Начальное состояние s_0 определяется ключом и синхропосылкой; на каждом шаге

$$y_i = f(s_{i-1}, x_i),\ s_i = h(s_{i-1}, x_i) \qquad (2.10)$$

Корректность локальна: f должна быть инъективна по входному знаку при каждом фиксированном состоянии. Частный случай — гаммирование: состояние движется независимо от текста,

$$y_i = x_i \oplus \gamma_i \qquad (2.11)$$

Модель естественна для поточных шифров и аппаратных реализаций.

Назначение. Модель задаёт поточную обработку данных знак за знаком: связывает ключ и синхропосылку с начальным состоянием автомата, сводит корректность к локальному требованию инъективности функции выходов по знаку и служит рабочим языком для построения и анализа генераторов гаммы, в том числе аппаратных реализаций поточных шифров.

Пример. Регистр сдвига с обратной связью (LFSR): состояние — 4-битный вектор, на каждом шаге выходной бит складывается со знаком текста, состояние сдвигается.

Вывод

Алгебраическая модель даёт условие корректности, которое в автоматной модели спускается до инъективности f по знаку; вероятностная модель задаёт критерий стойкости, которого требуют принципы построения поточных систем (неотличимость гаммы от случайной при однократном использовании); эффективность гаммирования достигается отказом от блочной структуры — ценой отказа от идеала совершенной секретности, недостижимого без условия (2.7).

Дополнительно была разработана визуальная интерактивная схема сравнения моделей блочных шифров с примерами (рисунок 2) на ***js, html, css*** *для закрепления материала. Название файла:* **2q_formal_models_of_ciphers**

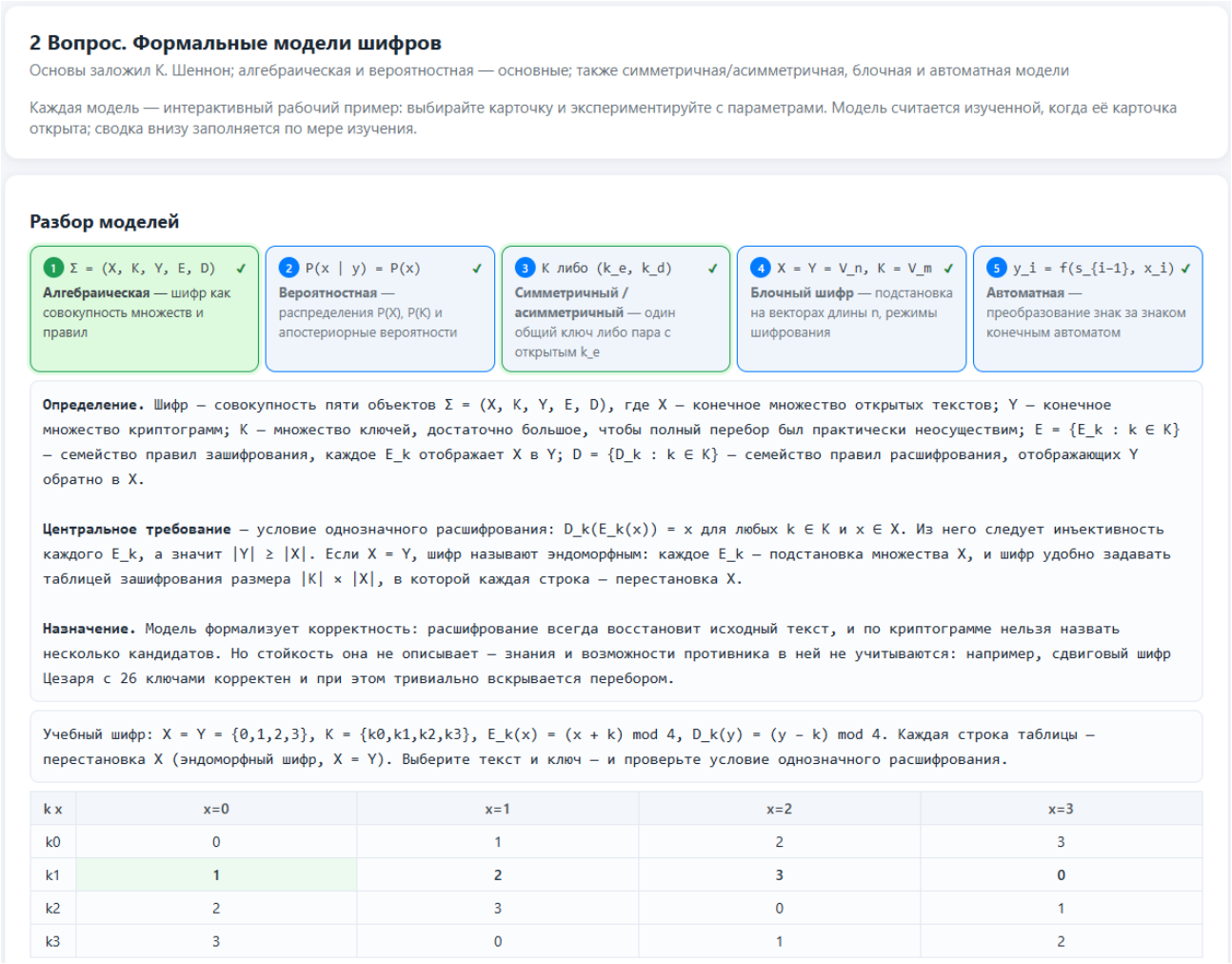


Рисунок 2 – Визуальная интерактивная схема сравнения моделей шифров.

3. Принципы построения поточных систем шифрования

Определение. Поточный шифр — система шифрования, в которой текст преобразуется знак за знаком: генератор вырабатывает ключевой поток (гамму), и каждый знак текста складывается с очередным знаком гаммы по модулю 2:

$$y_i = x_i \oplus \gamma_i,\ x_i = y_i \oplus \gamma_i \qquad (3.1)$$

где x_i — знак открытого текста; y_i — знак шифртекста; γ_i — знак ключевого потока (гаммы).

В терминах автоматной модели это конечный автомат (I, S, O, h, f), состояние которого движется независимо от текста, а начальное состояние s_0 определяется ключом и синхропосылкой. Стойкость шифра определяется свойствами гаммы: она должна вести себя как случайная последовательность,

известная только законным сторонам. При этом поточные шифры делятся на синхронные, где гамма не зависит от открытого текста, и самосинхронизирующиеся, где очередной знак гаммы зависит от предыдущих знаков шифртекста; ниже речь идёт прежде всего о синхронных, как основном классе. Принципы построения группируются вокруг четырёх целей: стойкость, корректность, эффективность и надёжность; в литературе они систематизированы в виде критериев Рюппеля — длинные периоды, большая линейная сложность, хорошие статистические свойства, диффузия статистики открытого текста.

Принципы стойкости

1. Длинный период гаммы — стойкость. Гамма вырабатывается детерминированным генератором, поэтому зацикливается. При коротком периоде разные участки сообщения шифруются одной гаммой, а перебор состояний генератора становится осуществимым. Для регистра сдвига длины L максимальный период равен $2^L - 1$ и достигается только примитивным многочленом обратной связи — так и строят генераторы. Пример: 4-битный LFSR с примитивной обратной связью даёт период $2^4 - 1 = 15$, проходя по кругу все ненулевые состояния.

2. Статистическая неотличимость гаммы от случайной — стойкость. Если гамма неравновероятна или её знаки зависимы, частотный и корреляционный анализ шифртекста накапливает сведения о тексте и состоянии генератора. Требование формализуют постулаты Голомба: количество единиц и нулей различается не более чем на единицу, распределение серий знаков соответствует случайному, автокорреляция двузначна. Пример: чистый LFSR этим постулатам удовлетворяет лишь частично, и потому слаб — его свойства восстанавливаются по короткому участку гаммы.

3. Нелинейность выработки гаммы — стойкость. Выход не должен быть линейной функцией состояния. Мерой служит линейная сложность последовательности — длина минимального РСЛОС, который её порождает: по алгоритму Берлекампа–Месси регистр с малой линейной сложностью восстанавливается по нескольким знакам гаммы, после чего вся последовательность вычисляется вперёд и назад. Пример: фрагмент гаммы часто известен из типичного заголовка сообщения — при линейном генераторе этого достаточно для взлома. Поэтому практические схемы ставят над регистрами нелинейные фильтры и комбинаторы нескольких регистров.

4. Однократность гаммы — стойкость (условие модели). Связано с шифром Вернама: совершенная секретность достигается лишь при однократной равновероятной гамме длины текста, что и составляет часть условия совершенной секретности. При повторе гаммы для двух текстов

$$y \oplus y' = x \oplus x' \tag{3.2}$$

— гамма исчезает, пара шифртекстов превращается в шифр простой замены: частотный анализ восстанавливает оба текста, а затем и саму гамму. Поэтому длина вырабатываемой гаммы всегда берётся не меньше длины сообщения.

Принципы корректности и инициализации

5. Синхропосылка — корректность и безопасность. Задаёт начальное состояние s_0. Назначение двоякое: во-первых, стороны должны выработать одинаковую гамму (иначе расшифрование даст бессмыслицу), во-вторых, разные сеансы с одним ключом должны давать разные гаммы — синхропосылка заменяет смену ключа. Правила: она не должна повторяться при одном ключе и не обязана быть секретной — всю секретность несёт ключ. Пример: повтор сеанса с тем же ключом и синхропосылкой порождает запрещённый повтор гаммы из принципа 4.

6. Обратимость и синхронизация — корректность. При каждом фиксированном состоянии функция выходов f инъективна по входному знаку (для сложения по модулю 2 это выполнено автоматически), поэтому расшифрование однозначно восстанавливает текст. Любое расхождение состояний сторон разрушает весь последующий поток, поэтому протоколы передачи включают механизмы ресинхронизации.

Принципы эффективности

7. Высокая скорость выработки гаммы — эффективность. Главное требование к поточному шифру — быстродействие, соизмеримое с поступлением данных: шифрование идёт знак за знаком без буферизации блоков, стоимость знака — несколько операций (сдвиг, сложение по модулю 2). Поэтому гаммирование — рабочая схема для скоростных каналов и встроенных систем, где блочные шифры проигрывают по производительности.

8. Экономичность ключа — эффективность и практичность. Генератор вырабатывает длинную гамму из короткого ключа: состояние занимает десятки–сотни байт, а стойкость обеспечивается не длиной ключа (как в Вернаме, где $|K| \geq |X|$), а вычислительной трудностью восстановления состояния. Это сознательный компромисс: отказ от идеала совершенной секретности в пользу вычислительной стойкости.

Принципы надёжности

9. Ошибкоустойчивость — надёжность. В синхронном шифровании гамма не зависит от текста, поэтому отсутствует эффект размножения ошибок: число искажённых знаков в расшифровке равно числу искажённых в шифртексте. Это выгодно отличает его от блочных схем со сцеплением, где искажение распространяется на весь блок. Самосинхронизирующиеся схемы, напротив, размножают ошибки, но сами восстанавливаются после потери синхронизации

— выбор класса определяется требованиями канала. При необходимости защиты от подмены знаков добавляют имитовставку.

Вывод

Группа 1–4 реализует критерии Рюппеля и, в пределе, требование вероятностной модели: гамма имитирует однократно используемую случайную последовательность. Принципы 5–6 обеспечивают корректность: синхропосылка гарантирует одинаковый и не повторяющийся от сеанса к сеансу ключевой поток без смены мастер-ключа, а обратимость и ресинхронизация — восстановление потока после расстройства. Группа 7–8 объясняет существование поточных шифров наряду с блочными: отказ от идеала совершенной секретности в обмен на вычислительную стойкость при минимальной цене знака. Принцип 9 описывает работу в реальных условиях — шум канала и подмену знаков, которые чистые модели не рассматривают.

4. Режим шифрования, в котором биективность всех шифрующих отображений базового режима не является необходимой для однозначного расшифрования криптограмм

Ответ: режим гаммирования с обратной связью (CFB); подойдут также гаммирование по счётчику (CTR) и OFB. Во всех этих режимах базовое блочное преобразование E_k используется только в прямом направлении, поэтому его биективность не требуется: достаточно, чтобы E_k было вычислимой функцией с ключом.

Почему биективность не требуется

Разберём CFB. Перед первым блоком задаётся синхропосылка y_0 . Зашифрование и расшифрование i-го блока выглядят так:

$$y_i = x_i \oplus E_k(y_{i-1}) \quad (4.1)$$

$$x_i = y_i \oplus E_k(y_{i-1}) \quad (4.2)$$

где x_i и y_i — i-е блоки открытого текста и криптограммы, y_0 — синхропосылка, E_k — блочное преобразование с ключом k.

Ключевое наблюдение: обратная функция E_k^{-1} в расшифровании не участвует вовсе. Получатель восстанавливает x_i , повторяя то же самое вычисление, что делал шифратор: цепочка обратной связи y_{i-1} состоит из уже принятых знаков криптограммы (или известной синхропосылки) и не зависит от открытого текста. Поэтому и шифратор, и дешифратор вычисляют одно и то же значение $E_k(y_{i-1})$), после чего гамма «стирается» сложением по модулю 2.

Сравните с режимами, где биективность действительно необходима. В ECB и CBC расшифрование опирается на обратное преобразование; в CBC это выглядит так:

$$x_i = E_k^{-1}(y_i) \oplus y_{i-1} \quad (4.3)$$

Если E_k не биективна — два разных входа дают одинаковый выход, — то E_k^{-1} не существует, а два разных открытых блока неразличимы по криптограмме: однозначное расшифрование невозможно в принципе.

Механизм однозначного расшифрования в CFB

1. Получатель накапливает шифртекст: ему доступны y_0 (синхропосылка) и принятые блоки y_i, y_2, …

2. Для каждого блока он вычисляет гамму $E_k(y_{i-1})$) — тем же ключом и по тем же аргументам, что и отправитель; обращение E_k не требуется.

3. Восстанавливает $x_i = y_i \oplus E_k(y_{i-1})$) по формуле (4.2). Для фиксированного y_{i-1} отображение $x_i \mapsto y_i$ — прибавление фиксированной величины по модулю 2; оно обратимо автоматически, поэтому пара (y_{i-1}, y_i) определяет x_i однозначно.

Отсюда вывод: однозначность расшифрования в CFB обеспечивается структурой режима — обратной связью по криптограмме и симметричностью сложения по модулю 2, а не свойствами базового преобразования. Даже если E_k выродится в константу, расшифрование останется однозначным — пострадает стойкость, но не корректность.

Усиление: гаммирование по счётчику (CTR и OFB)

То же верно для гаммирования по счётчику, где зависимость от шифртекста вообще отсутствует:

$$y_i = x_i \oplus E_k(s + i \bmod 2^n) \quad (4.4)$$

$$x_i = y_i \oplus E_k(s + i \bmod 2^n) \quad (4.5)$$

где s — начальное значение счётчика (синхропосылка), n — разрядность блока в битах.

Здесь E_k применяется только к значениям счётчика, известным обеим сторонам, и используется исключительно в прямом направлении: при расшифровании блочный шифр никогда не «расшифровывается». Единственное требование к E_k — детерминированность и вычислимость; биективность не входит в условия корректности режима.

5. Распространение ошибки в режиме гаммирования с обратной связью по шифртексту (CFB)

Постановка задачи. Сколько 64-битовых блоков открытого текста искажается при расшифровании криптограммы симметричного блочного шифра, если в шифрованном тексте исказился один бит и используется режим гаммирования с обратной связью по шифртексту (CFB), в котором блоки открытого текста имеют длину 64 бита, а блоки (сегменты) шифрованного текста — 8 бит?

Ответ: при расшифровании искажаются ровно два 64-битовых блока открытого текста. Обоснование ниже.

Логика работы режима CFB (64-битовый регистр, 8-битовые сегменты)

В шифраторе и дешифраторе работает одинаковый сдвиговый регистр длиной 64 бита, перед началом работы заполненный синхропосылкой $y_0 = \text{IV}$. Открытый текст разбивается на 64-битовые блоки, а каждый блок — на восемь 8-битовых сегментов; шифртекст передаётся посегментно.

На каждом шаге шифратор выполняет три действия: 1) вычисляет гамму $z_i = \text{E}_k(R_i)$, где R_i — текущее содержимое регистра; 2) берёт старшие (крайние левые) 8 битов гаммы и складывает их с очередным сегментом открытого текста: $y_i = x_i \oplus z_i$; 3) сдвигает регистр на 8 битов влево и дописывает в освободившиеся младшие разряды сам шифртекст y_i.

Дешифратор выполняет те же три действия: поскольку регистр пополняется шифртекстом, а не открытым текстом, получатель вычисляет ту же гамму и восстанавливает сегмент открытого текста по формуле

$$x_i = y_i \oplus \text{E}_k(R_i), \quad i = 1, 2, \ldots \qquad (5.1)$$

где R_i — содержимое регистра обратной связи на шаге i; E_k — блочное преобразование с ключом k.

Обратное преобразование E_k^{-1} в расшифровании не участвует вовсе — этим CFB отличается от режимов типа ECB и CBC (ответ к вопросу 4): режим работает даже при необратимом базовом преобразовании, достаточно его вычислимости.

Распространение ошибки: шаг за шагом

Пусть в шифртексте исказился один бит в сегменте y_j (позиция бита внутри сегмента — $1 \leq t \leq 8$). Последствия разворачиваются в два этапа.

Этап 1 — прямое влияние (сегмент j). При расшифровании $x_j = y_j \oplus z_j$ инвертированный бит шифртекста инвертирует ровно один бит открытого текста в сегменте j. Гамма z_j на этом шаге не пострадала: она зависит только от содержимого регистра, а регистр пока не содержит ошибочных битов.

Этап 2 — влияние через регистр (сегменты $j, j+1, \ldots, j+8$). Искажённый сегмент y_j дописывается в регистр и остаётся в нём, пока регистр не сдвинется на всю свою длину. Длина регистра — 64 бита, сдвиг за один шаг — 8 битов, поэтому бит ошибки циркулирует в регистре

$$\frac{64}{8} = 8 \qquad (5.2)$$

шагов. Всё это время бит ошибки входит в блок данных, подаваемый на вход E_k ; в силу лавинного эффекта выработанная гамма z_i полностью отличается от правильной, и весь соответствующий сегмент открытого текста (8 битов) восстанавливается неверно. После восьми шагов бит ошибки вытесняется из регистра, и расшифрование снова идёт правильно — режим CFB самосинхронизируется.

Итого при расшифровании затронуты девять последовательных сегментов — $j, j+1, \ldots, j+8$: один бит — непосредственно в сегменте j и по восемь битов в каждом из восьми следующих сегментов. Число искажённых битов открытого текста

$$1 \leq \Delta \leq 65 \qquad (5.3)$$

где Δ — число искажённых битов открытого текста. Верхняя граница $1 + 8 \cdot 8 = 65$ достигается, когда каждая из восьми поражённых гамм искажает все восемь битов своего сегмента; в пределе (если часть поражённых гамм совпадает с правильными) искажённым может оказаться только один прямой бит. В среднем искажается около 33 битов (1 + 8·4).

Пересчёт в 64-битовые блоки

Один 64-битовый блок открытого текста содержит восемь 8-битовых сегментов; сегменты блока B нумеруются по формуле

$$8(B - 1) + 1 \leq j \leq 8B \qquad (5.4)$$

где B — порядковый номер 64-битового блока (B = 1, 2, …).

Искажённые сегменты $j, j + 1, \ldots, j + 8$ — девять подряд идущих сегментов. Отсюда два свойства:

— минимум два блока: девять сегментов не помещаются в один блок, так как в нём всего восемь сегментов, — следовательно, хотя бы два блока затронуты всегда;

— максимум два блока: интервал из девяти последовательных сегментов пересекает не более одной границы между блоками; чтобы задеть три блока, пришлось бы пересечь две границы, а между соседними границами лежат ровно восемь сегментов — интервал длины девять не может накрыть обе.

Следовательно, искажения всегда попадают ровно в два 64-битовых блока открытого текста: в одном из них поражён начальный непрерывный участок сегментов, в другом — остальные.

Иллюстрация

Пусть бит исказился в сегменте $j = 10$ — это второй сегмент второго 64-битового блока (сегменты 9–16 образуют блок 2, сегменты 17–24 — блок 3). Характер искажений показан в таблице 1.

Таблица 1 — Распространение ошибки при j = 10

Сегмент	9	10	11	12	13	14	15	16	17	18
Блок текста	2	2	2	2	2	2	2	2	3	3
Искажено битов	—	1 бит	8 бит	8 бит	8 бит	8 бит	8 бит	8 бит	8 бит	8 бит

Затронуты блок 2 (частично — сегменты 10–16) и блок 3 (частично — сегменты 17–18): ровно два блока. Если бы ошибка попала в последний сегмент блока, $j = 16$, искажёнными оказались бы сегменты 16 и 17–24 — снова два блока: последний сегмент блока 2 и первые семь сегментов блока 3.

Вывод

При расшифровании криптограммы в режиме CFB с 64-битовым регистром обратной связи, 8-битовыми сегментами шифртекста и 64-битовыми блоками открытого текста одиночная ошибка в шифртексте искажает ровно два 64-битовых блока открытого текста: один бит — непосредственно в текущем сегменте и по одному полному 8-битовому сегменту в восьми следующих сегментах, пока бит ошибки циркулирует в регистре (формулы (5.2) и (5.3)). Размножение ошибки ограничено длиной регистра плюс один сегмент, после чего расшифрование восстанавливается автоматически — это типовое свойство самосинхронизирующихся режимов. Ответ на вопрос задания: искажаются два блока.

6. Коллизия случайной пары сообщений при заданной вероятности

Постановка задачи. Однонаправленная хэш-функция h принимает значения из множества двоичных n-мерных векторов, то есть $\{0,1\}^n$, мощность которого $N = 2^n$. Требуется найти минимальное n, при котором случайная пара сообщений m_1 и m_2 составляет коллизию, то есть $h(m_1) = h(m_2)$, с вероятностью, не большей 10^{-9}.

Вероятностная модель (парадокс дней рождений). Применим классическую модель задачи о днях рождения: N равновозможных значений «дней» и k независимых случайных выбора. Вероятность того, что все k значений попарно различны, равна N(N−1)…(N−k+1)/N^k, поэтому вероятность хотя бы одного совпадения составляет:

$$P_k = 1 - \frac{N(N-1)\cdots(N-k+1)}{N^k} \qquad (6.1)$$

где N — мощность множества значений хэш-функции, равная 2^n; k — число выбранных случайных сообщений.

При $k \ll N$ справедливо приближение, из которого следует знаменитый вывод парадокса дней рождений: коллизия возникает уже при числе сообщений порядка $\sqrt{N}$, а не N:

$$P_k \approx 1 - e^{\frac{-k(k-1)}{2N}} \qquad (6.2)$$

В рассматриваемом вопросе фигурирует одна случайная пара сообщений, поэтому полагаем $k = 2$ и получаем точное значение:

$$P_2 = 1 - \frac{N(N-1)}{N^2} = \frac{1}{N} = 2^{-n} \qquad (6.3)$$

где при упрощении числителя учтено, что $N^2 - N(N-1) = N$.

Неравенство и выражение n. Условие задачи задаёт неравенство для найденной вероятности (6.3):

$$2^{-n} \leq 10^{-9} \qquad (6.4)$$

Прологарифмируем обе части по основанию 2. Логарифм по основанию 2 — возрастающая функция, поэтому знак неравенства сохраняется:

$$-n \leq -9 \cdot \log_2 10 \qquad (6.5)$$

Умножив обе части на −1 и сменив знак неравенства на противоположный, выразим n:

$$n \geq 9 \cdot \log_2 10 \quad (6.6)$$

Вычисления с промежуточными шагами. Шаг 1. Найдём $\log_2 10$ через натуральные логарифмы:

$$\log_2 10 = \frac{\ln 10}{\ln 2} = \frac{2{,}302585}{0{,}693147} \approx 3{,}321928 \quad (6.7)$$

Шаг 2. Подставим найденное значение в неравенство (6.6):

$$n \geq 9 \cdot 3{,}321928 = 29{,}897352 \qquad (6.8)$$

Шаг 3. Поскольку n — целое число бит, а $29{,}897352 < 30$, минимальное значение, удовлетворяющее неравенству, — $n = 30$.

Проверка. Проверим обе стороны границы:

$$2^{-30} = \frac{1}{1\,073\,741\,824} \approx 9{,}31 \cdot 10^{-10} \leq 10^{-9} \qquad (6.9)$$

— условие задачи выполнено;

$$2^{-29} = \frac{1}{536\,870\,912} \approx 1{,}86 \cdot 10^{-9} > 10^{-9} \quad (6.10)$$

— при n = 29 условие нарушено. Следовательно, n = 30 действительно минимальное значение: при меньшей длине хэш-значения требуемая вероятность не обеспечивается.

Ответ: $n = 30$.

Замечание: следствие парадокса дней рождений. Подчеркнём различие двух постановок. Для одной случайной пары сообщений достаточно n = 30. Если же коллизию целенаправленно ищет противник, перебирая M сообщений, то по формуле (2) вероятность хотя бы одной коллизии:

$$P_M \approx 1 - e^{\frac{-M(M-1)}{2^{n+1}}} \approx \frac{M(M-1)}{2^{n+1}} \qquad (6.11)$$

При $n = 30$ заметная вероятность достигается уже при $M \approx 1{,}17 \cdot 2^{\frac{n}{2}} \approx 38\,600$ сообщений (ведь $2^{15} \approx 32\,768$). Именно поэтому криптографические хэш-функции имеют значительно большую длину выхода: для SHA-256, где $n = 256$, граница дней рождений 2^{128} недостижима на современных вычислителях.